

FAST · arXiv 2501.09747 · Physical Intelligence

Empiricist

11+3 · seat 2 · WEIGHT 2

Tokenizer smoke + AR vs FM

HF FAST+ · Table I · latency · P0-P1

Oct 19 paper club · VLA Architectures

MYTH BUST: FAST $\neq$ FM · $\pi_{0.5}$ = FAST pretrain $\rightarrow$ FM deploy

$\pi_0 \approx \text{FM}$ · $\pi_0\text{-FAST} \approx \text{discrete AR}$ · $\pi_{0.5} \approx \text{hybrid successor}$

CRITICAL MYTH BUST · say this first

FAST does NOT replace flow matching at deploy

Label	Meaning
FAST	Discrete DCT→BPE action tokenizer for AR VLAs
FM	Continuous flow-matching (π_0 / $\pi_{0.5}$ deploy)
π_0 -FAST	Discrete AR variant using this tokenizer
$\pi_{0.5}$	FAST for pretrain · FM for post-train/deploy

Naming: $\pi_0 \approx \text{FM}$ · π_0 -FAST $\approx$ discrete AR · $\pi_{0.5} \approx$ hybrid successor

FAST pipeline · DCT → BPE

Quantile norm → DCT → scale γ → round → low-freq-first flatten → BPE $|V|=1024$

① Norm

1-99% quantile
→ [-1,1]

② DCT

per-dim
over H

③ Quant

$\text{round}(\gamma \cdot C)$
 $\gamma=10$ default

④ Flatten

low-freq
across dims first

⑤ BPE

vocab 1024
~30 tok/arm

Why it works

- ▶ Naïve bins at high Hz → copy-last-token
- ▶ DCT packs shape into low-freq coeffs
- ▶ Sparse zeros → BPE squashes stream
- ▶ Invertible decode → continuous actions
- ▶ Overwrite least-used VLM vocab slots

Defaults / HF

Chunk $\approx$ 1 second

$\gamma = 10$ · $|V| = 1024$

Pad dims to 32 (FAST+)

```
AutoProcessor.from_pretrained(  
    "physical-intelligence/fast",  
    trust_remote_code=True)
```

HF FAST+ smoke recipe

physical-intelligence/fast · $\gamma=10$ · $|V|=1024$

```
from transformers import AutoProcessor

tok = AutoProcessor.from_pretrained(
    "physical-intelligence/fast", trust_remote_code=True)

tokens = tok(action_chunk)  # quantile-normed 1s chunks

# custom: new_tok = tok.fit(action_dataset)
```

Must log

tokens/chunk
recon MSE vs γ
wall ms encode

Defaults

$\gamma = 10$
 $|V| = 1024$
chunk = 1 s

Sanity

Bridge~20 DROID~29
bussing~28 shirt~53
vs naïve 35/105/140/700

Sanity Table I · tokens / 1 s chunk

Comparable reconstruction · Empiricist smoke target

Dataset	Dim	Hz	Naïve	FAST	×
BridgeV2	7	5	35	~20	1.75×
DROID	7	15	105	~29	3.6×
Bussing	7	20	140	~28	5.0×
Shirt Fold	14	50	700	~53	13.2×

FAST tends ~30 tokens/arm/1s (~60 bi-manual) — largely independent of Hz

Fail if your encode does not land near these orders of magnitude at matched recon MSE

Inference latency · RTX 4090 class

~100 ms FM vs ~750 ms AR-FAST per 1 s chunk (~7.5×)

π_0 FM / diffusion

~100 ms

~10 FM steps × ~300M expert

π_0 -FAST AR

~750 ms

~30-60 AR tokens × full ~2-3B LM

Static manip OK · dynamic closed-loop / eval throughput hurt

Future: speculative decode · quant · kernels · Empiricist must log ms/chunk

Empiricist smoke matrix · priority grid

P0 Y · P1 AR-FAST vs continuous FM · Defer full 10k h

P	Experiment	Smoke
P0	HF FAST+ encode/decode; compression vs naïve; didactic AR toy across Hz	Y
P1	VLA FT FAST+ vs bins; BPE on/off; γ sweep; AR-FAST vs continuous FM	Partial
P2	fit() vs universal; $\pi_{0.5}$ hybrid echo (short FAST→FM)	Partial
Defer	Full π_0 -FAST 10k h; real laundry hardware	N

Env outline: /workspace/fast-quiet/env_outline/

Log: tokens/chunk · recon MSE · train steps · success · ms/chunk · GPU name · max_memory_allocated

Empiricist hypotheses H1-H6

H1 · High-Hz failure

Naïve bins degrade as Hz↑; FAST stays flat

H2 · Compression

≥2× fewer tokens @ matched MSE on ≥1500

H3 · BPE

DCT-only beats naïve; full FAST wins SR + length

H4 · Train vs FM

AR-FAST fewer steps on large mix; tie on tiny

H5 · Infer latency

FM ms/chunk ≪ AR-FAST (100 vs 750 class)

H6 · Hybrid

FAST pre→FM post beats always-FM or always-AR

When FAST helps vs when continuous FM wins

FAST helps

- ▶ High-Hz next-token prediction
- ▶ Train compute / compression
- ▶ Denser chunks without token blowup
- ▶ Language-following AR VLAs
- ▶ Cheap large-scale discrete pretrain
- ▶ $\pi_{0.5}$ discrete pretrain phase

Continuous FM wins

- ▶ Inference latency / smooth dynamics
- ▶ Closed-loop control (fewer ms/chunk)
- ▶ Real-time household deploy
- ▶ ~ 100 ms vs ~ 750 ms on 4090
- ▶ π_0 / $\pi_{0.5}$ post-train & serve
- ▶ Contact-rich multimodal density

$\pi_{0.5}$ recipe: FAST for efficient pretrain NTP → serve with continuous FM

Env outline path · no jobs launched

/workspace/fast-quiet/env_outline/

conda_env.yaml

PyTorch + transformers for FAST+

setup_mamba.sh

create env; OpenPI/OpenVLA pointers

slurm_tokenizer_smoke.sh

encode/decode + γ sweep

slurm_toy_ar_hz.sh

Fig. 3 didactic AR vs Hz

slurm_ft_fast_vs_naive.sh

short VLA FT FAST vs naïve

slurm_fast_vs_fm.sh

matched AR-FAST vs continuous FM

run_checklist.md

GPU honesty preflight

Fig.4 DCT→BPE pipeline + Alg.1

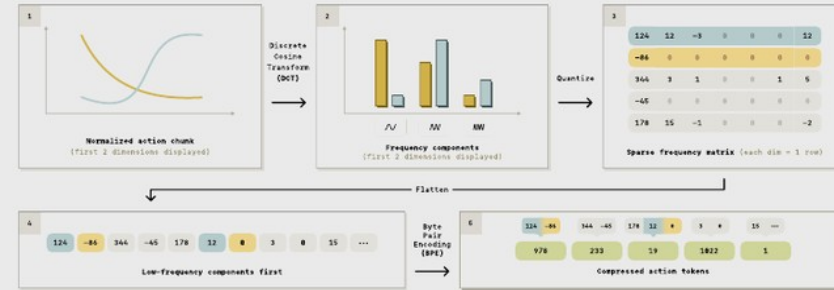


Fig. 4: Overview of the FAST action tokenization pipeline. Given a normalized chunk of actions, we apply discrete cosine transform (DCT) to convert the signal to the frequency domain. We then quantize the DCT coefficients and use byte-pair encoding (BPE) to compress the flattened sequence of per-dimension DCT coefficients into the final action token sequence. See Section V-B for a detailed description.

matrix into a sequence of dense tokens. We flatten the matrix into a 1-dimensional vector of integers, interleaving action dimensions by including all low-frequency components first, and train a byte pair encoding (BPE) tokenizer [27] to losslessly compress it into dense action tokens. The BPE step “squashes” the zero-valued components and merges frequently-occurring coefficient combinations across action dimensions. We choose BPE to compress the DCT matrix, since many efficient implementations exist and it can produce a fixed-size output vocabulary that can be easily integrated into the existing vocabulary of vision-language models for VLA training. Other lossless compression algorithms like Huffman coding [33] or Lempel-Ziv methods [75] (the algorithms underlying the gzip compression approach) could be used instead, but we leave this investigation for future work.

Note that the *order* of flattening the $|A| \times H$ DCT coefficient matrix prior to BPE encoding can have significant impact on policy training. There are two options: column-first flattening, i.e., concatenate the lowest-frequency components for each dimension first, or row-first flattening, i.e., concatenating all frequency components for a single action dimension first. We choose the former, since we find that predicting the *low-frequency* components, that characterize the overall shape of the output sequence, first during autoregressive prediction leads to more stable policy rollouts.

All operations in our tokenization pipeline are easily invertible, allowing fast decoding of predicted actions. The tokenizer has only two hyperparameters: the scale applied to the DCT coefficients before rounding, and the vocabulary size of the BPE compression step. We find that both parameters are not very sensitive, and we use the same values across all our single-dataset tokenization experiments (rounding scale 10, BPE vocabulary size 1024). This is in contrast to end-to-end *learned* compression modules that rely on vector quantiza-

Algorithm 1 FAST Tokenizer

Require: scale γ , (for inference) BPE dictionary Φ
procedure FASTTOKENIZER($a_{1:H}$)
 $C_1^i \leftarrow \text{DCT}(a_{1:H}^i)$ $\triangleright$ Compute DCT coefficients
 $C_j^i \leftarrow \text{round}(\gamma \cdot C_j^i)$ $\triangleright$ Quantize coefficients
 $[T_k] \leftarrow [C_1^1, C_2^1, \dots, C_1^H, \dots, C_H^H]$ $\triangleright$ Flatten tokens
BPE Training:
 $\phi \leftarrow \text{TrainBPE}(\mathcal{D} := \{[T_k]\})$
Tokenization:
 $[T_1, \dots, T_k] \leftarrow \text{BPE}([T_1, \dots, T_k], \phi)$
return action_tokens

tion [59]. Such networks are often tedious to train, and require careful dataset-specific hyperparameter selection to achieve good reconstruction [66, 48]. Our experiments show that our DCT-based tokenization approach trains higher-performing policies than VQ-based approaches, while being significantly simpler and easier to tune.

We empirically demonstrate the benefits of our DCT-based tokenization in the toy example from Section IV. Figure 3 shows that training the autoregressive model on DCT-compressed target tokens achieves constantly low prediction error across a wide range of sampling frequencies. We provide a concise summary of our tokenization approach in Algorithm 1 and test the effectiveness of FAST tokenization on robot control problems in Section VI.

C. A Universal Robot Action Tokenizer

The only *learned* component of our tokenizer is the vocabulary of the BPE encoder, which needs to be trained for each new dataset that the tokenizer is being applied to. While this learning process is fast (typically only a few minutes), it adds additional friction to using FAST tokenization. Thus,

approach that allows us to train the first autoregressive VLAs on dexterous and high-frequency robot data.

Action representations for VLA training. Prior works have explored various action parameterizations for training robot policies, including VLAs. One line of work uses “semantic” action representations like language sub-tasks [21, 2, 4], or keypoints [50, 32, 25, 19]. Such approaches can often learn from few examples or even perform tasks *zero-shot* without any robot examples [50, 32, 25], but require hand-designed low-level controllers for task execution, limiting their generality. An alternative approach directly trains VLAs to output low-level robot control commands given image and language instruction inputs. The most common design directly embeds actions into discrete tokens, that can be generated with standard autoregressive sequence models, like any popular vision-language model. Existing approaches map from continuous robot actions to discrete action tokens using a simple per-dimension, per-timestep binning scheme [9, 10, 39]. We find that this scheme struggles to scale to high-frequency robot control tasks. We propose a new tokenization scheme for robot actions, based on time-series compression techniques, that allows us to train autoregressive VLAs on high-frequency data. A number of works have also proposed alternatives to tokenization, for example by using regression heads or introducing new weights for diffusion decoding [20, 7, 41, 63]. In comparison, our approach does not require modifications of the underlying pre-trained transformer model, can easily be applied to any pre-trained autoregressive transformer model, and achieves competitive performance to state-of-the-art diffusion-based VLAs [7] across many tasks, while being significantly more compute efficient to train.

Another set of related work explores *vector-quantized* action representations [41, 3, 49]. Such approaches train a vector-quantized encoder-decoder network, for which reconstruction quality can be sensitive to hyperparameter choices and structure [66]. We find that these methods perform well at coarse, low-fidelity reconstruction tasks, but fail on high-frequency tasks when fine-grained control is required. In comparison, our FAST tokenization scheme has few hyperparameters and can reconstruct actions with high precision while offering strong compression properties.

III. PRELIMINARIES

Problem formulation. Our goal is to train policies $\pi(a_{1:H}|o)$ that map an observation o to a sequence of future robot actions $a_{1:H}$. We assume that policies output an “action chunk” [69, 40], a *sequence* of H actions [15, 7, 69], which makes it easier to produce temporally-consistent actions and reduces compounding error. The goal of *action tokenization* is to define a mapping $\mathcal{T}_a : a_{1:H} \rightarrow [T_1, \dots, T_n]$ from a sequence of continuous actions $a_{1:H}$, with dimensionality $|A|$, to a sequence of n discrete tokens $T \in |\mathcal{V}|$ from a vocabulary of size $|\mathcal{V}|$. Note that the number of tokens n may differ between action sequences, just like sentences of the same length may be tokenized into a variable number of text tokens.

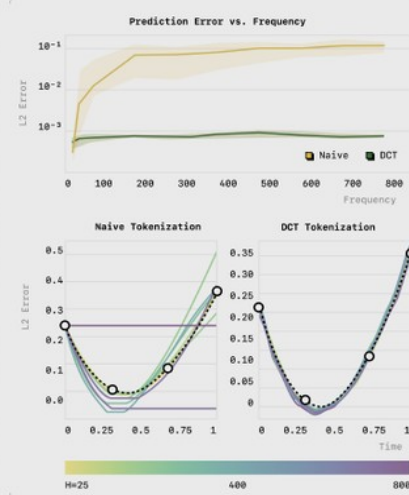


Fig. 3: **Effect of sampling rate on prediction performance.** We train a small autoregressive transformer model on a didactic interpolation task, in which the network must predict the black dashed curve given the four circles. We find that models trained with the binning tokenization approach used in prior VLAs [10, 39] produce increasingly poor predictions as we increase the sampling frequency of the underlying signal, due to strong correlation between consecutive tokens at high frequencies. Our FAST tokenization approach, based on the discrete cosine transform (DCT), addresses the problem and leads to high-quality predictions across all sampling rates.

Binning-based action tokenization. The most commonly used approach for action tokenization is a simple binning discretization scheme [8, 10, 39, 72, 56]. For a given action a , this approach discretizes each dimension independently, dividing the range of values in the training dataset into N uniform bins, most commonly using $N = 256$. For a *sequence* of D -dimensional actions $a_{1:H}$, this tokenization scheme would be applied to each time step, resulting in a final token sequence $\mathcal{T}_a(a_{1:H}) = [T_{1,1}, \dots, T_{1,D}, \dots, T_{H,1}, \dots, T_{H,D}]$. For high-frequency robot data, this tokenization scheme is sub-optimal: it can easily produce hundreds of tokens per action chunk, which make training challenging and lead to slow inference.

IV. CASE STUDY: HOW DOES TOKENIZATION AFFECT VLA TRAINING?

To illustrate the challenge of training autoregressive policies with current action tokenization approaches, we start

on dexterous and high-frequency robot data.

Action representations for VLA training. Prior works have explored various action parameterizations for training robot policies, including VLAs. One line of work uses “semantic” action representations like language sub-tasks [21, 2, 4], or keypoints [50, 32, 25, 19]. Such approaches can often learn from few examples or even perform tasks *zero-shot* without any robot examples [50, 32, 25], but require hand-designed low-level controllers for task execution, limiting their generality. An alternative approach directly trains VLAs to output low-level robot control commands given image and language instruction inputs. The most common design directly embeds actions into discrete tokens, that can be generated with standard autoregressive sequence models, like any popular vision-language model. Existing approaches map from continuous robot actions to discrete action tokens using a simple per-dimension, per-timestep binning scheme [9, 10, 39]. We find that this scheme struggles to scale to high-frequency robot control tasks. We propose a new tokenization scheme for robot actions, based on time-series compression techniques, that allows us to train autoregressive VLAs on high-frequency data. A number of works have also proposed alternatives to tokenization, for example by using regression heads or introducing new weights for diffusion decoding [20, 7, 41, 63]. In comparison, our approach does not require modifications of the underlying pre-trained transformer model, can easily be applied to any pre-trained autoregressive transformer model, and achieves competitive performance to state-of-the-art diffusion-based VLAs [7] across many tasks, while being significantly more compute efficient to train.

Another set of related work explores *vector-quantized action*

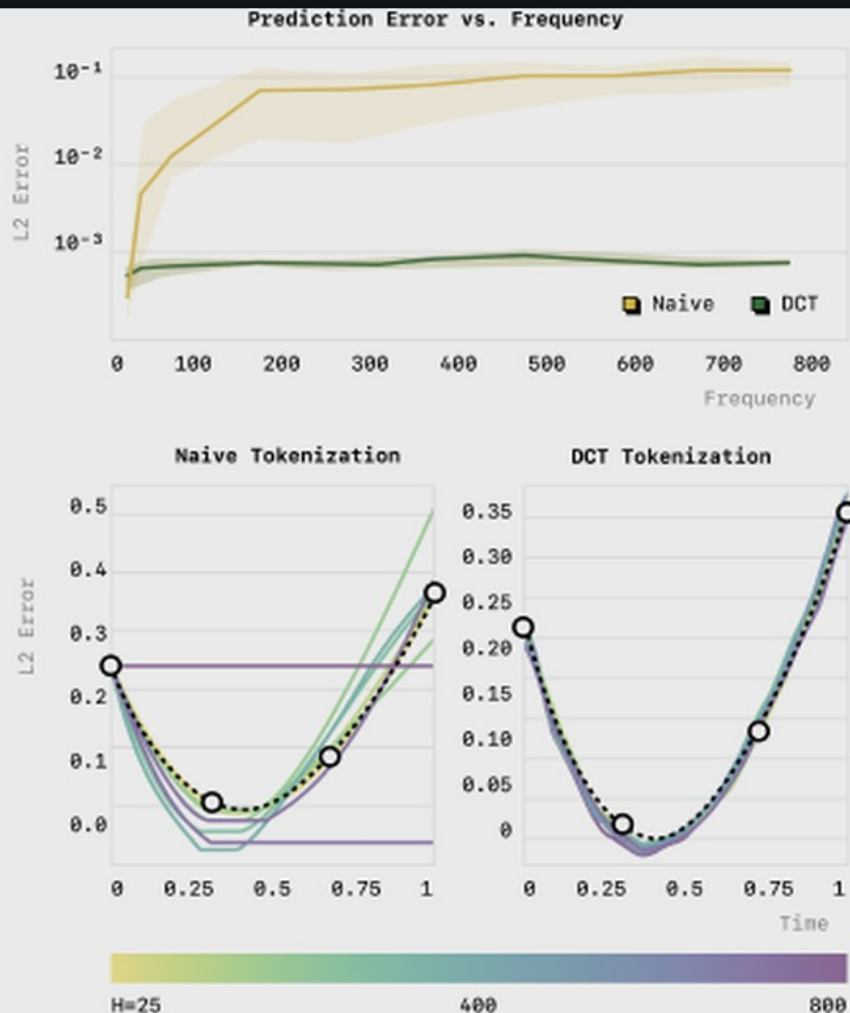


Fig. 3: Effect of sampling rate on prediction performance. We train a small autoregressive transformer model on a didactic interpolation task, in which the network must predict the black dashed curve given the four circles. We find that models trained with the binning tokenization approach used in prior VLAs [10, 39] produce increasingly poor predictions as

Fig.9 FAST vs diffusion π_0

which the tokenizer is applied to a completely new robot setup without recomputing the tokenization. We find that the FAST+ tokenizer achieves good compression performance across a wide range of robot datasets, reducing the number of action tokens by 2x across all datasets, and significantly more on some.

We also test performance of the universal tokenizer for policy training, and report results alongside the per-dataset tokenizers in Figure 6. Across all tasks, the *universal* tokenizer closely matches the performance of the dataset-specific FAST tokenizers, suggesting that the universal tokenizer can be used as a strong default for robot action tokenization.

D. Ablation Studies

We analyze two key aspects of our method: (1) Is our FAST tokenization approach *independent* of the underlying VLA backbone? (2) How important is the BPE compression step, the only learned component of our tokenization pipeline.

To answer the first question, we train an OpenVLA policy [39] on the challenging high-frequency T-shirt folding dataset, comparing the naïve tokenization approach originally used in OpenVLA to our FAST+ tokenizer. To comply with the task setup, we modify the OpenVLA model code to accept multiple input images and predict 1-second action chunks. The results on the right demonstrate that FAST is able to significantly boost performance of OpenVLA, enabling it to train effectively on high-frequency robot manipulation data. This suggests, that our tokenization approach is *independent* of the underlying model backbone, and may be easily applied to a wide range of pre-trained autoregressive transformer models.

Secondly, we ablate the BPE encoding step on the table bussing and T-shirt folding tasks. The figure on the right shows that the resulting policies *without BPE encoding* achieve worse rollout performance (but still outperform naïve tokenization). Intuitively, the DCT transform still concentrates most of the signal's information in a few tokens, improving the learning signal. However, without BPE, there is a large number of repeated 0-tokens which dilute the learning signal and also significantly slow down inference, since models need to autoregressively predict hundreds of action tokens, ultimately leading to worse policy performance.

E. Comparing FAST to Diffusion

In this section, we compare π_0 , a state-of-the-art diffusion VLA, to our model that combines π_0 with FAST and uses



Fig. 9: Comparison of diffusion π_0 [7] to our π_0 model with FAST decoding on single-task training. On small datasets (Liberio, T-Shirt Folding), both perform comparably. On large datasets (Table Bussing), FAST converges faster. In DROID, we find that FAST follows language instructions better. We report mean and 95% CI.

autoregressive decoding. We compare the performance of both models on the tasks from Section VI-B.

We report results in Figure 9. We find that on small datasets (Liberio, T-Shirt Folding; <50h), both VLAs perform comparably. However, on large datasets like Table Bussing, we find that the FAST-based VLA converges significantly faster, reaching high performance with 3x fewer training steps than the diffusion variant of π_0 . Additionally, we find that the autoregressive π_0 model trained with FAST tokenization follows language instructions more closely: in the DROID evaluations, the diffusion π_0 model often ignores the language instructions, leading to a lower score. We will leave a detailed investigation of the language following abilities of diffusion and autoregressive VLAs to future work.

One current limitation of the autoregressive VLA is its inference speed: while π_0 with diffusion typically predicts one second action chunks within 100ms on an NVIDIA 4090 GPU, the π_0 model with FAST tokenization needs approximately 750ms of inference time per chunk, since it must perform more autoregressive decoding steps (typically 30-60 action tokens need to be decoded, vs. 10 diffusion steps for diffusion π_0) and use the full 2B parameter language model backbone for autoregressive decoding (vs. a 300M parameter “action expert” for diffusion π_0). While we did not find this slower inference to hurt performance on the static manipulation tasks we evaluated, it made evaluations significantly slower. Going forward, there are many techniques for accelerating the inference of discrete, autoregressive transformer models that are used extensively in the LLM literature (e.g., speculative decoding, quantization, custom inference kernels, etc.), but we will leave an investigation of these to future work.

Empiricist · close

P0: HF FAST+ smoke + Table I + didactic Hz. P1: AR-FAST vs continuous FM.

Defer full 10k h · Quote (tokens, MSE, ms/chunk, GPU) with every score

Q&A · Oct 19 · FAST / 2501.09747

Sanity Table I · tokens / 1 s chunk

Comparable reconstruction · Empiricist smoke target

Dataset	Dim	Hz	Naïve	FAST	×
BridgeV2	7	5	35	~20	1.75×
DROID	7	15	105	~29	3.6×
Bussing	7	20	140	~28	5.0×
Shirt Fold	14	50	700	~53	13.2×

FAST tends ~30 tokens/arm/1s (~60 bi-manual) — largely independent of Hz

Fail if your encode does not land near these orders of magnitude at matched recon MSE